

一个虚表。

用户可以用 SQL 对视图和基本表进行查询。在用户眼中，视图和基本表都是关系，用户可以在视图上再定义视图。

本书从 3.2 节起，将逐一介绍 SQL 语句的功能和语法。为了突出基本概念和基本功能，将会略去许多语法细节。不同关系数据库管理系统产品在实现标准 SQL 时各有差别，与 SQL 标准的符合程度也不相同(一般符合的百分比在 85% 以上)。因此，读者在使用时还应参阅具体产品的用户手册。

3.2 数据定义

本节介绍 SQL 的数据定义功能，包括数据库模式、表、视图和索引的定义等。SQL 的数据定义语句如表 3.3 所示。

表 3.3 SQL 的数据定义语句

操作对象	操作方式		
	创建	删除	修改
数据库模式	CREATE SCHEMA	DROP SCHEMA	* SQL 标准无修改语句
表	CREATE TABLE	DROP TABLE	ALTER TABLE
视图	CREATE VIEW	DROP VIEW	
索引	* CREATE INDEX	* DROP INDEX	* ALTER INDEX

SQL 标准没有提供修改数据库模式定义的语句。用户如果想修改此对象，只能先将它删除然后再重建。SQL 标准也没有提供索引相关的语句，但为了提高查询效率，商用关系数据库管理系统通常都提供了索引机制和相关的语句，如表 3.3 中创建、删除和修改索引等。

在早期的数据库系统中，所有数据库对象都属于一个数据库，也就是说只有一个命名空间。所以在第 1 章图 1.15 中一个数据库只对应一个内模式、一个模式。

当前的关系数据库管理系统提供了层次结构的数据库对象命名机制，如图 3.2 所示。一个关系数据库管理系统的实例可以建多个数据库，一个数据库中又可以建多个模式，一个模式下通常包含多个表、视图和索引等数据库对象。



图 3.2 数据库对象命名机制的层次结构

本节仅介绍如何定义模式、基本表和索引，视图的概念及其定义方法将在 3.6 节专门讨论。

3.2.1 模式的定义与删除

1. 定义模式

在 SQL 中，定义模式语句如下：

```
CREATE SCHEMA [ <模式名> ] AUTHORIZATION <用户名>;
```

如果没有指定<模式名>，那么<模式名>隐含为<用户名>。

要创建模式，调用该命令的用户必须拥有数据库管理员 (DBA) 权限，或者获得了 DBA 授予的 CREATE SCHEMA 权限。

[例 3.1] 为用户 WANG 定义一个“学生选课”模式 S-C-SC。

```
CREATE SCHEMA "S-C-SC" AUTHORIZATION WANG;
```

注意：标识符要么加双引号，要么没有任何引号。这里"S-C-SC"是加了双引号的。

[例 3.2] 未指定模式名的模式定义示例。

```
CREATE SCHEMA AUTHORIZATION WANG;
```

该语句没有指定<模式名>，所以<模式名>隐含为用户名“WANG”。

定义模式实际上定义了一个命名空间，在这个空间中可以进一步定义该模式包含的数据库对象，如基本表、视图、索引等。

这些数据库对象可以用表 3.3 中相应的 CREATE 语句来定义。

目前，在 CREATE SCHEMA 中可以接受 CREATE TABLE、CREATE VIEW 和 GRANT 子句。也就是说，用户可以在创建模式的同时在该模式定义中进一步创建基本表、视图，定义授权。即

```
CREATE SCHEMA <模式名> AUTHORIZATION <用户名>  
[ <表定义子句> | <视图定义子句> | <授权定义子句> ];
```

[例 3.3] 为用户 ZHANG 创建一个模式 Test，并且在其中定义一个表 Tab1。

```
CREATE SCHEMA Test AUTHORIZATION Zhang  
CREATE TABLE Tab1 ( Col1 SMALLINT,  
                     Col2 INT,  
                     Col3 CHAR(20),  
                     Col4 NUMERIC(10,3),  
                     Col5 DECIMAL(5,2)  
);
```

2. 删除模式

在 SQL 中，删除模式语句如下：

```
DROP SCHEMA <模式名> [ CASCADE | RESTRICT ];
```

其中 CASCADE 和 RESTRICT 两者必选其一。选择了 CASCADE(级联),表示在删除模式的同时把该模式中所有的数据库对象全部删除。选择了 RESTRICT(限制),表示如果该模式中已经定义了数据库对象(如表、视图等),则拒绝该删除语句的执行;只有当该模式中没有任何数据库对象时才能执行 DROP SCHEMA 语句。

[例 3.4] 删除例 3.3 中建立的模式 Test。

```
DROP SCHEMA Test CASCADE;
```

该语句删除了模式 Test,同时也将该模式中已经定义的表 Tab1 删除了。

3.2.2 基本表的定义、删除与修改

1. 定义基本表

创建了一个模式就建立了一个数据库的命名空间,一个框架。在这个空间中首先要定义的是该模式包含的数据库基本表。

SQL 使用 CREATE TABLE 语句定义基本表,其基本格式如下:

```
CREATE TABLE <表名>(<列名><数据类型>[列级完整性约束]
    [,<列名><数据类型>[列级完整性约束]]
    ...
    [,<表级完整性约束>]);
```

建表的同时,通常还可以定义与该表有关的完整性约束,这些完整性约束被存入系统的数据字典。当用户操作表中数据时,由关系数据库管理系统自动检查该操作是否违背这些完整性约束。如果完整性约束涉及该表的多个属性列,则必须定义在表级上,否则既可以定义在列级也可以定义在表级。

本章以“学生选课”数据库为例来讲解 SQL 语句。为此,首先要定义一个“学生选课”模式 S-C-SC,该模式中包含以下三个表。

- ① “学生”表: Student(Sno, Sname, Ssex, Sbirthdate, Smajor)。
- ② “课程”表: Course(Cno, Cname, Ccredit, Cpno)。
- ③ “学生选课”表: SC(Sno, Cno, Grade, Semester, Teachingclass)。

注意:关系的主码加下划线表示。

这三个基本表的定义如下,其示例数据可参见二维码内容(即第 2 章图 2.1)。

[例 3.5] 建立一个“学生”表 Student。

```
CREATE TABLE Student
    (Sno CHAR(8) PRIMARY KEY, /* 列级完整性约束条件,Sno 是主码 */
    Sname VARCHAR(20) UNIQUE, /* Sname 取唯一值 */
    Ssex CHAR(6),
    Sbirthdate Date,
```



示例数据:学生选课
模式 3 个表的示例
数据

```
Smajor VARCHAR(40)
);
```

系统执行该 CREATE TABLE 语句后,就在数据库中建立一个新的空“学生”表 Student,并将该表及有关约束的定义存放在数据字典中。

[例 3.6] 建立一个“课程”表 Course。

```
CREATE TABLE Course
( Cno CHAR(5) PRIMARY KEY,           /* 列级完整性约束,Cno 是主码 */
  Cname VARCHAR(40) NOT NULL,        /* 列级完整性约束,Cname 不能取空值 */
  Ccredit SMALLINT,
  Cpno CHAR(5),
  FOREIGN KEY (Cpno) REFERENCES Course(Cno)
  /* 表级完整性约束,Cpno 是外码,被参照表是 Course,被参照列是 Cno */
);
```

参照表和被参照表可以是同一个表。Cpno 是 Cno 课程的直接先修课,并且只列出一门直接先修课^①。

[例 3.7] 建立“学生选课”表 SC。

```
CREATE TABLE SC
( Sno CHAR(8),
  Cno CHAR(5),
  Grade SMALLINT,                    /* 成绩 */
  Semester CHAR(5),                  /* 开课学期 */
  Teachingclass CHAR(8),             /* 学生选修某一门课所在的教学班 */
  PRIMARY KEY(Sno,Cno),
  /* 主码由两个属性构成,必须作为表级完整性进行定义 */
  FOREIGN KEY(Sno) REFERENCES Student(Sno),
  /* 表级完整性约束,Sno 是外码,被参照表是 Student */
  FOREIGN KEY(Cno) REFERENCES Course(Cno)
  /* 表级完整性约束,Cno 是外码,被参照表是 Course */
);
```

2. 数据类型

关系模型中一个很重要的概念是域。每一个属性来自一个域,它的取值必须是域中的值。在 SQL 中域的概念用数据类型来实现。定义表的各个属性时需要指明其数据类型及长度。SQL 标准支持多种数据类型,表 3.4 列出了几种常用的数据类型。要注意,不同的关系数据库

^① 实际情况会复杂些,一门课程可能会有几门直接先修课,我们将在第 7 章数据库设计中进一步讲解。

管理系统中支持的数据类型会有细微差别，读者使用时要参考具体产品的手册。

表 3.4 SQL 标准常用的数据类型

数据类型	含义
CHAR(<i>n</i>), CHARACTER(<i>n</i>)	长度为 <i>n</i> 的定长字符串
VARCHAR(<i>n</i>), CHARACTERVARYING(<i>n</i>)	最大长度为 <i>n</i> 的变长字符串
CLOB	字符串大对象
BLOB	二进制大对象
INT, INTEGER	整数(4 字节), 取值范围是 $[-2\,147\,483\,648, 2\,147\,483\,647]$
SMALLINT	短整数(2 字节), 取值范围是 $[-32\,768, 32\,767]$
BIGINT	大整数(8 字节), 取值范围是 $[-2^{63}, 2^{63}-1]$
NUMERIC(<i>p</i> , <i>d</i>)	定点数, 由 <i>p</i> 位数字(不包括符号、小数点)组成, 小数点后面有 <i>d</i> 位数字
DECIMAL(<i>p</i> , <i>d</i>), DEC(<i>p</i> , <i>d</i>)	同 NUMERIC 类似, 但数值精度不受 <i>p</i> 和 <i>d</i> 的限制
REAL	取决于机器精度的单精度浮点数
DOUBLE PRECISION	取决于机器精度的双精度浮点数
FLOAT(<i>n</i>)	可选精度的浮点数, 精度至少为 <i>n</i> 位数字
BOOLEAN	逻辑布尔量
DATE	日期, 包含年、月、日, 格式为 YYYY-MM-DD
TIME	时间, 包含一日的时、分、秒, 格式为 HH:MM:SS
TIMESTAMP	时间戳类型
INTERVAL	时间间隔类型

一个属性选用哪种数据类型要根据实际情况来决定。一般从两个方面来考虑, 即取值范围和要做的运算。例如, 对于学生选修某一门课程的成绩(Grade)属性, 可以采用 CHAR(3)作为数据类型, 但考虑到要在成绩上做算术运算(如求平均成绩), 而 CHAR(*n*)数据类型不能进行算术运算, 所以采用整数作为数据类型。整数又有大整数、整数和短整数三种, 通常课程的成绩采用百分制, 所以选用短整数作为成绩的数据类型。

3. 模式与表

每一个基本表都属于某一个模式, 一个模式包含多个基本表。当定义基本表时一般可以有三种方法定义它所属的模式。例如, 在例 3.1 中定义了一个“学生选课”模式 S-C-SC。现在要在该模式中定义 Student、Course、SC 等基本表。

方法一：在表名中明显地给出模式名。

```
CREATE TABLE "S-C-SC". Student (...); /* Student 所属的模式是 S-C-SC */  
CREATE TABLE "S-C-SC". Course (...); /* Course 所属的模式是 S-C-SC */  
CREATE TABLE "S-C-SC". SC (...); /* SC 所属的模式是 S-C-SC */
```

方法二：在创建模式语句中同时创建表，如例 3.3 所示。

方法三：设置所属的模式，这样在创建表时表名中不必给出模式名。

当用户创建基本表(其他数据库对象也一样)时若没有指定模式，系统根据搜索路径来确定该对象所属的模式。

搜索路径包含一组模式列表，关系数据库管理系统会使用模式列表中第一个存在的模式作为数据库对象的模式名。若搜索路径中的模式名都不存在，系统将给出错误提示。

使用下面的语句可以显示当前的搜索路径：

```
SHOW SEARCH_PATH;
```

搜索路径的当前默认值是“\$user, PUBLIC”，表示首先搜索与用户名相同的模式名，如果该模式名不存在，则使用 PUBLIC 模式。

DBA 也可以设置搜索路径，例如：

```
SET SEARCH_PATH TO "S-C-SC", PUBLIC;
```

然后，定义基本表：

```
CREATE TABLE Student (...);
```

实际结果是建立了 S-C-SC. Student 基本表。因为关系数据库管理系统发现搜索路径中第一个模式名是 S-C-SC，就把该模式作为基本表 Student 所属的模式。

4. 修改基本表

随着应用环境和应用需求的变化，有时需要修改已建好的基本表。SQL 用 ALTER TABLE 语句修改基本表，其一般语法如下：

```
ALTER TABLE <表名>  
[ADD[ COLUMN]<新列名><数据类型>[完整性约束]]  
[ADD <表级完整性约束>]  
[DROP[ COLUMN]<列名>[CASCADE| RESTRICT]]  
[DROP CONSTRAINT<完整性约束名>[RESTRICT | CASCADE ]]  
[RENAME COLUMN <列名> TO <新列名>]  
[ALTER COLUMN <列名> TYPE <数据类型>];
```

其中：

① <表名>是要修改的基本表。

② ADD 子句用于增加新列、新的列级完整性约束和新的表级完整性约束。

③ DROP COLUMN 子句用于删除表中的列，如果指定了 CASCADE 短语，则自动删除引用了该列的其他对象，比如视图；如果指定了 RESTRICT 短语，则如果该列被其他对象引用，关系数据库管理系统将拒绝删除该列。

④ DROP CONSTRAINT 子句用于删除指定的完整性约束。

⑤ RENAME COLUMN 子句用于修改列名。

⑥ ALTER COLUMN 子句用于修改列的数据类型。

[例 3.8] 向 Student 表增加“邮箱地址”列 Semail，其数据类型为字符型。

```
ALTER TABLE Student ADD Semail VARCHAR(30);
```

注意：不论基本表中原来是否已有数据，新增加的列一律为空值。

[例 3.9] 将 Student 表中出生日期 Sbirthdate 的数据类型由 DATE 型改为字符型。

```
ALTER TABLE Student ALTER COLUMN Sbirthdate TYPE VARCHAR(20);  
/* DATE 类型占用 19 字节,所以修改为 VARCHAR 类型时长度要大于或等于 19 */
```

[例 3.10] 增加课程名称必须取唯一值的约束条件。

```
ALTER TABLE Course ADD UNIQUE(Cname);
```

5. 删除基本表

当某个基本表不再使用时，可以使用 DROP TABLE 语句将其删除。其一般格式为：

```
DROP TABLE <表名>[RESTRICT | CASCADE];
```

同样，若选择 RESTRICT，则该表的删除有限制条件，即该表不能被其他表的约束所引用（如 CHECK、FOREIGN KEY 等约束），不能有视图，不能有触发器(trigger)，不能有存储过程或函数等。如果存在这些依赖该表的对象，则此表不能被删除。

若选择 CASCADE，则该表的删除没有限制条件。在删除基本表的同时，相关的依赖对象，都可能被一起删除。

该语句的默认选项是 RESTRICT。

[例 3.11] 删除 Student 表，选择 CASCADE。

```
DROP TABLE Student CASCADE;
```

执行该语句后，Student 基本表定义一旦被删除，不仅表中的数据和此表的定义将被删除，而且此表上建立的索引、约束、触发器等对象一般也都将被删除。有的关系数据库管理系统会同时删除在此表上建立的视图。如果欲删除的基本表被其他基本表所引用，则这些表相应的参照完整性约束也可能被级联删除。因此删除基本表的操作一定要格外小心。

[例 3.12] 删除 Student 表，若表上建有视图，选择 RESTRICT 时表不能删除；选择 CAS-

CADE 时可以删除表, 视图也自动被删除。

```
CREATE VIEW CS_Student /* 在 Student 表上建立计算机科学与技术专业的学生视图 */
AS
```

```
SELECT Sno, Sname, Ssex, Sbirthdate, Smajor
```

```
FROM Student
```

```
WHERE Smajor='计算机科学与技术';
```

```
DROP TABLE Student RESTRICT; /* 采用 RESTRICT 方式删除 Student 表 */
```

```
--ERROR: cannot drop table Student because other objects depend on it
```

```
/* 系统返回错误信息, 指出存在依赖该表的对象, 此表不能被删除 */
```

```
DROP TABLE Student CASCADE; /* 采用 CASCADE 方式删除 Student 表 */
```

```
--NOTICE: drop cascades to view CS_Student /* 系统返回提示, 此表上的视图也被删除 */
```

```
SELECT * FROM CS_Student; /* CS_Student 视图不存在 */
```

```
--ERROR: relation "CS_Student" does not exist
```

注意: 不同的数据库产品, 在遵循 SQL 标准的基础上, 其具体实现和处理 DROP TABLE 语句的策略会与标准有差别。所以, 在实际应用中一定要阅读相应的产品手册。



扩展阅读: 不同产品 DROP TABLE 处理策略比较

我们对比了 DROP TABLE 的 SQL 标准与 Kingbase ES、Oracle、MS SQL Server 三种数据库产品的不同处理策略。读者可以扫描二维码进行阅读。

3.2.3 索引的建立与删除

当表的数据量很大时, 查询操作会比较耗时。建立索引是加快查询速度的有效手段。数据库索引类似于图书的目录, 能快速定位到需要查询的内容。用户可以根据应用环境的需要在基本表上建立一个或多个索引, 以提供多种存取路径, 加快查找速度。

数据库索引有多种类型, 常见的索引结构包括顺序表(sequential list)索引、B+树索引、哈希索引(hash index)、位图索引(bitmap index)等。顺序表索引是针对按指定属性值升序或降序存储的关系, 在该属性上建立一个顺序索引文件, 索引文件由属性值和相应的元组指针组成。B+树索引是将索引属性组织成 B+树形式, 其叶结点为属性值和相应的元组指针。B+树索引具有动态平衡的优点。哈希索引是建立若干个桶, 将索引属性按照其哈希函数值映射到相应桶中, 桶中存放索引属性值和相应的元组指针。哈希索引具有查找速度快的特点。位图索引是用位向量记录索引属性中可能出现的值, 每个位向量对应一个可能值。第9章中将详细介绍索引技术。

索引虽然能够加速数据库查询, 但需要占用一定的存储空间, 当基本表更新时, 索引也要进行相应的维护, 这些都会增加数据库的负担, 因此要根据实际应用的需要有选择地创建索引。

目前 SQL 标准中没有涉及索引,但商用关系数据库管理系统一般都支持索引机制,只是不同的关系数据库管理系统支持的索引类型不尽相同。

一般说来,建立与删除索引由数据库管理员或表的属主(owner,即建立表的人)负责完成。关系数据库管理系统在执行查询时会自动选择是否使用索引或使用哪个索引作为存取路径,用户不必也不能自主地选择索引。索引是关系数据库管理系统的内部实现技术,属于内模式的范畴。

1. 建立索引

在 SQL 语言中,建立索引使用 CREATE INDEX 语句,其一般格式如下:

```
CREATE[UNIQUE][CLUSTER]INDEX <索引名>  
ON <表名>(<列名>[<次序>][,<列名>[<次序>]]...);
```

其中,<表名>是要建索引的基本表的名称。索引可以建立在该表的一列或多列上,各列名之间用逗号分隔。每个<列名>后面还可以用<次序>指定索引值的排列次序,可选 ASC(升序)或 DESC(降序),默认值为 ASC。

UNIQUE 表明此索引的每一个索引值只对应唯一的数据记录。

CLUSTER 表示要建立的索引是聚簇索引。一张表上只能建立一个聚簇索引,多个有联系的表可以建立聚簇索引。有关聚簇索引将在第 7 章 7.5.2 小节关系模式存取方法选择中进一步介绍。

[例 3.13]为“学生选课”数据库中的 Student、Course 和 SC 三个表建立索引。其中 Student 表按学生姓名升序建唯一索引, Course 表按课程名升序建唯一索引, SC 表按学号升序和课程号降序建唯一索引(即先按照学号升序,对同一个学号再按课程号降序)。

```
CREATE UNIQUE INDEX Idx_StuSname ON Student(Sname);  
/* 保证了 Sname 取唯一值的约束 */  
CREATE UNIQUE INDEX Idx_CouCname ON Course(Cname);  
/* 加上 Cname 取唯一值的约束 */  
CREATE UNIQUE INDEX Idx_SCCno ON SC(Sno ASC,Cno DESC);
```

2. 修改索引

对于已经建立的索引,如果需要对其重新命名,可以使用 ALTER INDEX 语句。其一般格式如下:

```
ALTER INDEX <旧索引名> RENAME TO <新索引名>;
```

[例 3.14]将 SC 表的 Idx_SCCno 索引名改为 Idx_SCSnoCno。

```
ALTER INDEX Idx_SCCno RENAME TO Idx_SCSnoCno;
```

3. 删除索引

索引一经建立就由系统使用和维护,无须用户干预。建立索引是为了减少查询操作的时间。

间,但如果数据更新(增、删、改)频繁,系统会花费许多时间来维护索引,从而降低了查询效率。这时可以删除一些不必要的索引。

在 SQL 中,删除索引使用 DROP INDEX 语句,其一般格式如下:

DROP INDEX <索引名>;

[例 3.15] 删除 Student 表的 Idx_StuSname 索引。

DROP INDEX Idx_StuSname;

删除索引时,系统会同时从数据字典中删去有关该索引的描述。

3.2.4 数据字典

数据字典是关系数据库管理系统内部的一组系统表,它记录了数据库中所有的定义信息,包括关系模式定义、视图定义、索引定义、完整性约束定义、各类用户对数据库的操作权限、统计信息等。关系数据库管理系统在执行 SQL 的数据定义语句时,实际上就是把这些定义信息存入系统的数据字典以更新其中的相应信息。在进行查询处理和查询优化时,关系数据库管理系统要根据数据字典中的信息执行处理算法和优化算法,因此数据字典是关系数据库管理系统运行的重要依据。

3.3 数据查询

数据查询是数据库的核心操作。SQL 提供了 SELECT 语句进行数据查询,该语句具有灵活的使用方式和丰富的功能。

SELECT 语句的一般格式如下:

```
SELECT[ ALL|DISTINCT]<目标列表达式>[别名][,<目标列表达式>[别名]]...  
FROM <表名或视图名>[别名][,<表名或视图名>[别名]]... | (<SELECT 语句>)[AS]<别名>  
[WHERE <条件表达式>]  
[GROUP BY <列名 1>[HAVING <条件表达式>]]  
[ORDER BY <列名 2>[ASC|DESC]]  
[LIMIT <行数 1>[OFFSET <行数 2>]];
```

SELECT 语句的含义是,根据 WHERE 子句的条件表达式从 FROM 子句指定的基本表、视图或派生表中找出满足条件的元组,再按 SELECT 子句中的目标列表达式选出元组中的属性值形成结果表。

如果有 GROUP BY 子句,则将结果按<列名 1>的值进行分组,该属性列值相等的元组为一个组,通常会在每组中作用聚集函数。如果 GROUP BY 子句带 HAVING 短语,则只有满足指定条件的组才予以输出。